

Lab 6: Likelihood Intervals

Instructor notes. Part 1: prep reading. Part 2: podium and circulating cheat sheet

How to prepare (for students)

- **Big picture.** On Monday we turned the width of a likelihood curve into a confidence interval. Today we add the shortcut most software uses (the curvature at the MLE, from the Hessian) and see what changes when a model has two parameters instead of one. The lab uses a classic dose-response experiment on stored-grain beetles.
- **Read:** Bolker 6.4.1 on likelihood slices and profiles (printed p. 249-254) and 6.5 on the quadratic approximation (p. 260-265).
- **Review:** Monday's likelihood ratio interval and the 1.92 cutoff; what a second derivative measures; the logistic (log-odds) model from the Week 5 WOD.
- **Bring:** a calculator for Act 1, and a laptop with R for Act 2 (you'll use `optim()` and `solve()`).

Overview (instructor only)

Beat	Time
Opening: admin plus the material moved from Monday	~25 min
Act 1 (paper): <i>Reading the Surface</i>	~45 min
Act 2 (laptop): <i>An Interval for the Lethal Dose</i>	~80 min

The opening carries two pieces that didn't fit in Monday's lecture: the quadratic approximation (one parameter, pikas) and slices vs. profiles with two parameters (Bolker's tadpoles). Keep the two pieces to about 15 minutes combined; Act 1 practices both on new data, so the opening only needs to introduce the ideas. With the longer opening, Act 2 is more likely to go home; it's written to work as a take-home.

Examples. Both opening examples use data other than the beetles, so nothing here gives away an Act answer. Two things to hold back:

- Don't derive NLL'' for the Poisson on the board; that's Wednesday's take-home WOD. Showing `optim()`'s value (2) is fine.
- Don't compare $1/\sqrt{H_{aa}}$ with the standard error from `solve(H)`. That comparison is Act 1, question 2a.

Answer keys: labs/answer_keys/Lab6_Act1_Key.Rmd and Lab6_Act2_Key.Rmd.

Part 1: Opening

A. The quadratic approximation (~8 min)

Pika haypiles from Monday: 2, 5, 3, 0, 4, 4; Poisson with mean λ ; MLE 3; likelihood ratio interval [1.82, 4.61].

```
haypiles <- c(2, 5, 3, 0, 4, 4)
lambda_hat <- mean(haypiles)

nll_pois <- function(lambda, y) {
  -sum(dpois(y, lambda = lambda, log = TRUE))
}

lambda_grid <- seq(0.01, 8, by = 0.01)
dnll <- sapply(lambda_grid, nll_pois, y = haypiles) - nll_pois(lambda_hat, haypiles)
lr_cut <- qchisq(0.95, df = 1) / 2
f_cut <- function(l) nll_pois(l, haypiles) - nll_pois(lambda_hat, haypiles) - lr_cut
lr_int <- c(uniroot(f_cut, c(0.5, 3))$root, uniroot(f_cut, c(3, 8))$root)
```

Key idea. Finding the likelihood ratio interval takes a search. The shortcut: near its minimum, the NLL looks like a parabola, and the width of a parabola is set by its second derivative.

A Taylor expansion around $\hat{\lambda}$ (Calc II), using the fact that the first derivative is zero at the minimum:

$$\text{NLL}(\lambda) \approx \text{NLL}(\hat{\lambda}) + \frac{1}{2} \text{NLL}''(\hat{\lambda}) (\lambda - \hat{\lambda})^2$$

Set the increase equal to the 1.92 cutoff and solve:

$$\frac{1}{2} \text{NLL}''(\hat{\lambda}) (\lambda - \hat{\lambda})^2 = 1.92 \Rightarrow \lambda = \hat{\lambda} \pm \frac{1.96}{\sqrt{\text{NLL}''(\hat{\lambda})}}$$

So the **standard error** is $1/\sqrt{\text{curvature}}$ (Bolker eq. 6.5.5). `optim(..., hessian = TRUE)` computes the curvature numerically:

```
fit <- optim(par = 2, fn = nll_pois, y = haypiles, method = "Brent",
            lower = 0.01, upper = 20, hessian = TRUE)
fit$hessian

##      [,1]
## [1,]    2
```

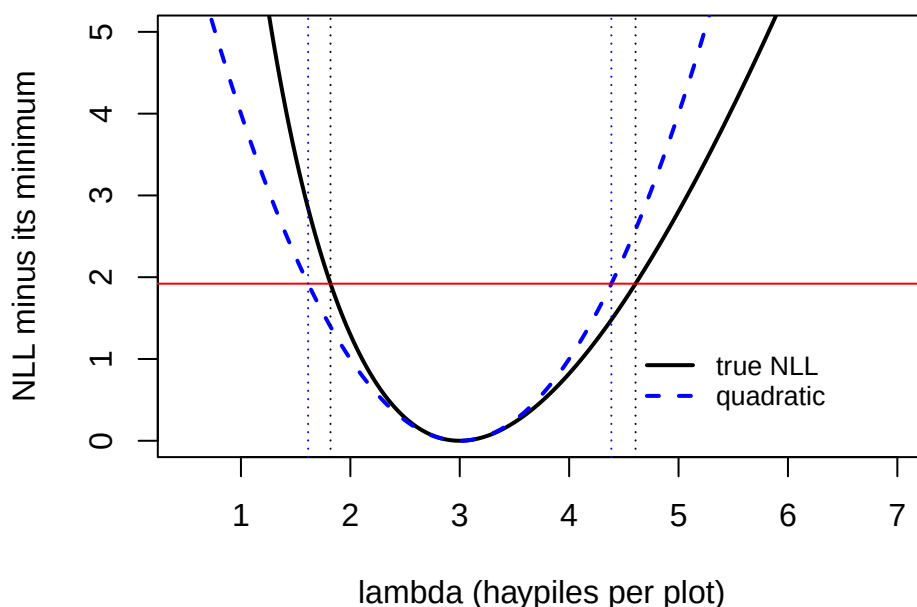
```

se_lambda <- 1 / sqrt(fit$hessian[1, 1])
wald_int <- fit$par + c(-1, 1) * qnorm(0.975) * se_lambda
round(c(se = se_lambda, lower = wald_int[1], upper = wald_int[2]), 2)

##      se lower upper
## 0.71  1.61  4.39

quad_approx <- 0.5 * fit$hessian[1, 1] * (lambda_grid - lambda_hat)^2
par(mar = c(4, 4.5, 0.5, 0.5))
plot(lambda_grid, dnll, type = "l", lwd = 2, xlim = c(0.5, 7), ylim = c(0, 5),
      xlab = "lambda (haypiles per plot)", ylab = "NLL minus its minimum")
lines(lambda_grid, quad_approx, lwd = 2, lty = 2, col = "blue")
abline(h = lr_cut, col = "red")
abline(v = lr_int, lty = 3)
abline(v = wald_int, lty = 3, col = "blue")
legend(x = 4.5, y = 1.3, c("true NLL", "quadratic"), lwd = 2, lty = c(1, 2),
      cex = 0.8, col = c("black", "blue"), bty = "n")

```



- Quadratic (Hessian) interval: [1.61, 4.39], symmetric by construction. Likelihood ratio: [1.82, 4.61], following the lopsided curve.
- With more data the NLL gets closer to a parabola and the two agree.
- **The practical default.** In most multiparameter models, the standard errors software reports (`summary()` of a `glm`, `lme4`, `glmmTMB`, and so on) come from this curvature, because it's cheap. The likelihood ratio interval is the check for key parameters and for small samples or estimates near a boundary.

Sticking point: why a *second* derivative? The first derivative is zero at the MLE, so it says nothing about width. The second derivative says how fast the slope picks up as you move

away, which is how fast the fit gets worse. Sketch a narrow and a wide parabola with the same vertex and ask which has the larger second derivative.

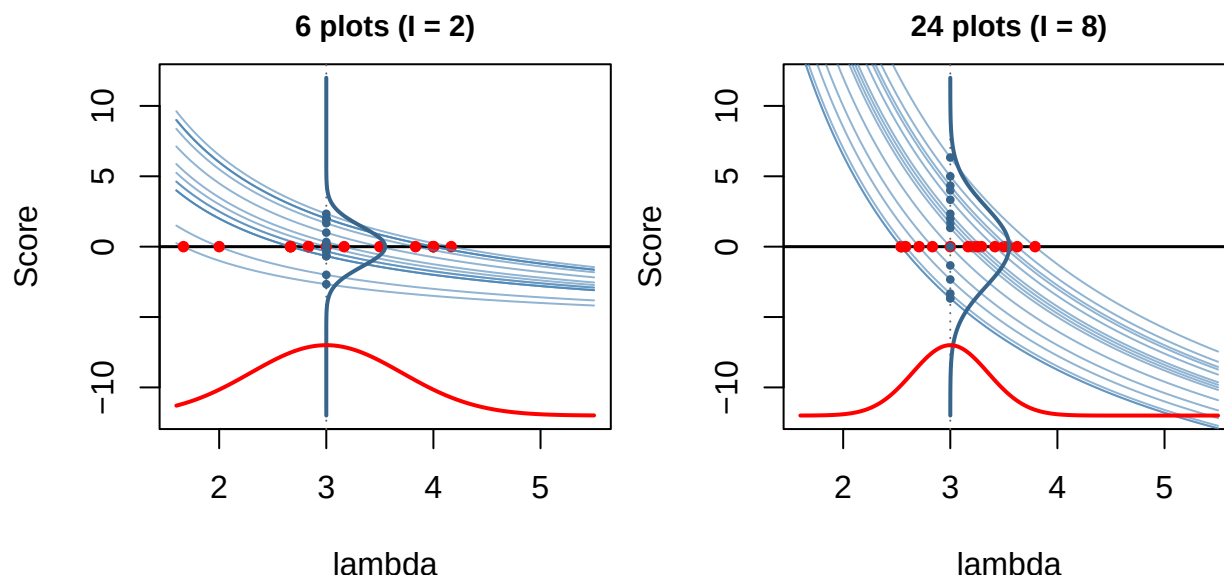
Aside (optional): why the MLE is approximately normal

Only if a student asks why the Hessian interval uses a normal distribution, or if the opening runs short. The argument works through the **score**, the slope of the log-likelihood, $U(\lambda) = \frac{d}{d\lambda} \log \mathcal{L}(\lambda)$. The MLE is where the score crosses zero. For the pikas, $U(\lambda) = \sum y_i / \lambda - n$.

```
set.seed(746)
lambda0 <- 3
lam <- seq(1.6, 5.5, length.out = 300)
score <- function(l, y) sum(y) / l - length(y)

score_panel <- function(n, n_sims = 15, ylim = c(-12, 12)) {
  info <- n / lambda0
  plot(NA, xlim = range(lam), ylim = ylim, xlab = "lambda", ylab = "Score",
       main = paste0(n, " plots (I = ", round(info, 1), ")"), cex.main = 0.9)
  abline(h = 0, lwd = 1.5)
  abline(v = lambda0, lty = 3, col = "grey40")
  for (i in seq_len(n_sims)) {
    y <- rpois(n, lambda0)
    lines(lam, sapply(lam, score, y = y), col = adjustcolor("steelblue", 0.6))
    points(mean(y), 0, pch = 16, col = "red", cex = 0.8)
    points(lambda0, score(lambda0, y), pch = 16, col = "steelblue4", cex = 0.6)
  }
  yy <- seq(ylim[1], ylim[2], length.out = 200)
  lines(lambda0 + 0.55 * dnorm(yy, 0, sqrt(info)) / dnorm(0, 0, sqrt(info)), yy,
        col = "steelblue4", lwd = 2)
  dx <- dnorm(lam, lambda0, sqrt(1 / info))
  lines(lam, ylim[1] + 5 * dx / max(dx), col = "red", lwd = 2)
}

par(mfrow = c(1, 2), mar = c(4, 4, 2, 0.5))
score_panel(6)
score_panel(24)
```



Each blue curve is the score from one simulated survey with true $\lambda = 3$. Build it up in steps:

1. **One curve.** The MLE is where it crosses zero (red dot).
2. **Many surveys.** Each survey shifts the curve up or down, so the crossing points (the MLEs) spread along the axis.
3. **Look up and down at the true value.** The heights there (blue dots) are centered on 0 and normal: each is a sum of independent contributions, one per plot, so the CLT applies. Their variance is the Fisher information I ($= n/\lambda$ here; blue bell).
4. **Vertical to horizontal.** Near the truth each curve is close to a line with slope $-I$. A curve at height h crosses zero about h/I away, so the MLEs have variance $I/I^2 = 1/I$ (red bell).
5. **More data.** With 24 plots the heights spread *more* (variance grows like n), but the curves get steeper faster (slope grows like n), so the crossings tighten like $1/\sqrt{n}$.

In the 6-plot panel the curves are bent rather than straight, so the crossings spread farther to the right than the left. That's the same lopsidedness as the likelihood ratio interval in section A, and it's where the normal approximation starts to fail.

B. More than one parameter (~7 min)

Bolker's tadpole data (Figures 6.7-6.8, in the reading): Holling type II predation, $p = a/(1 + ahN)$, binomial number killed.

```
library(emdbook)
data(ReedfrogFuncresp)
initial_n <- ReedfrogFuncresp$Initial
killed <- ReedfrogFuncresp$Killed

nll_holling <- function(a, h) {
  p <- a / (1 + a * h * initial_n)
```

```

    -sum(dbinom(killed, size = initial_n, prob = p, log = TRUE))
  }

tad_fit <- optim(c(0.5, 0.0125), function(par) nll_holling(par[1], par[2]),
               method = "L-BFGS-B", lower = c(0.01, 0.0001),
               upper = c(2, 0.1), hessian = TRUE)
a_hat <- tad_fit$par[1]
h_hat <- tad_fit$par[2]
min_nll <- tad_fit$value
round(tad_fit$par, 4)

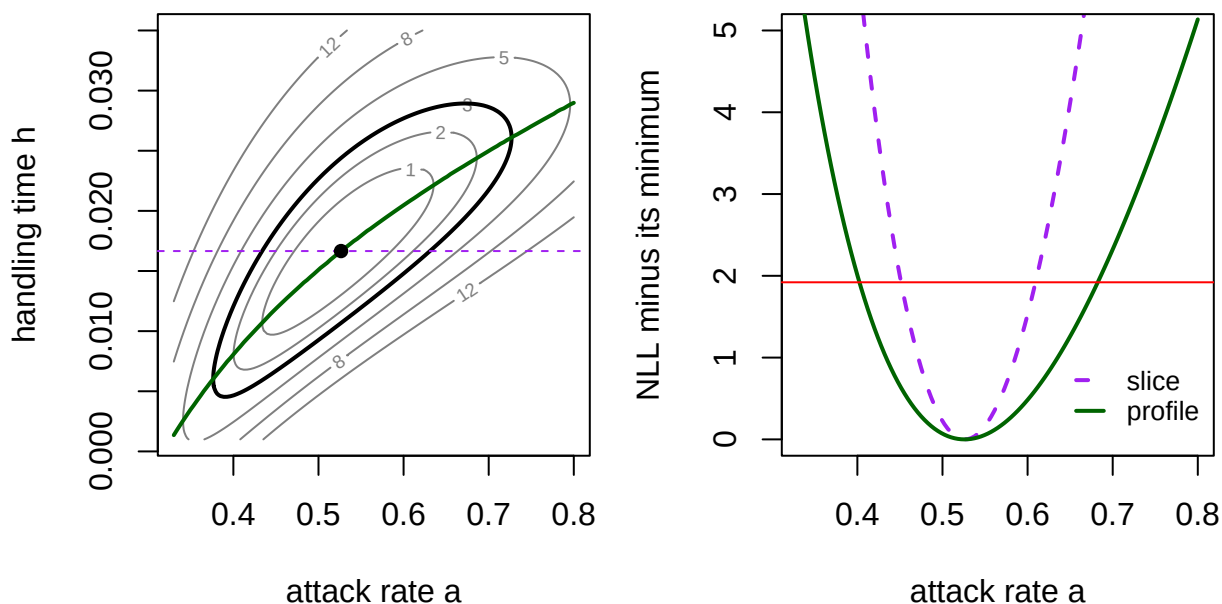
## [1] 0.5265 0.0167

a_seq <- seq(0.33, 0.8, length.out = 121)
h_seq <- seq(0.001, 0.035, length.out = 121)
nll_surf <- outer(a_seq, h_seq, Vectorize(nll_holling))

profile_h <- sapply(a_seq, function(a) {
  optimize(function(h) nll_holling(a, h), c(0.0001, 0.1))$minimum
})
profile_a <- sapply(seq_along(a_seq), function(i) nll_holling(a_seq[i], profile_h[i]))
slice_a <- sapply(a_seq, nll_holling, h = h_hat)

par(mfrow = c(1, 2), mar = c(4, 4.5, 0.5, 0.5))
contour(a_seq, h_seq, nll_surf - min_nll, levels = c(1, 2, 3, 5, 8, 12),
        xlab = "attack rate a", ylab = "handling time h", col = "grey50")
contour(a_seq, h_seq, nll_surf - min_nll, levels = qchisq(0.95, 2) / 2,
        add = TRUE, lwd = 2, drawlabels = FALSE)
abline(h = h_hat, lty = 2, col = "purple")
lines(a_seq, profile_h, lwd = 2, col = "darkgreen")
points(a_hat, h_hat, pch = 16)
plot(a_seq, slice_a - min_nll, type = "l", lwd = 2, lty = 2, col = "purple",
     ylim = c(0, 5), xlab = "attack rate a", ylab = "NLL minus its minimum")
lines(a_seq, profile_a - min_nll, lwd = 2, col = "darkgreen")
abline(h = lr_cut, col = "red")
legend(x = 0.63, y = 1.1, c("slice", "profile"), lwd = 2, lty = c(2, 1),
      cex = 0.8, seg.len = 1.2, col = c("purple", "darkgreen"), bty = "n")

```



- **Left:** the NLL surface is a tilted valley; estimates of a and h are correlated (a higher attack rate can be offset by a longer handling time).
- **Slice** (purple): hold h at $\hat{h}$ and vary a . **Profile** (green): for each a , re-optimize h ; it follows the floor of the valley.
- **Right:** the two curves give different intervals for a . Ask the room which they'd trust, and leave it open: Act 1, question 1a answers it on a much more tilted surface.

With two parameters, the Hessian is a 2×2 table of second derivatives. Inverting it gives the variance-covariance matrix:

```
vcov_tad <- solve(tad_fit$hessian)
round(sqrt(diag(vcov_tad)), 4) # standard errors of a and h

## [1] 0.0710 0.0049

round(cov2cor(vcov_tad), 2) # correlation between the estimates

##      [,1] [,2]
## [1,] 1.00 0.82
## [2,] 0.82 1.00
```

The correlation (0.82) is the tilt of the valley in one number. Stop here; Act 1 Part 2 works out what the off-diagonal entry does to a standard error.

Part 1 (continued): Act-by-Act notes

Act 1: Reading the Surface (paper, ~45 min)

Bliss's beetles, $\log \frac{p}{1-p} = a + bx$ with $x = \log_{10}$ dose. MLEs $\hat{a} = -60.72$, $\hat{b} = 34.27$.

- **1a.** Slice ≈ 34.1 to 34.4 (essentially a single point); profile ≈ 28.9 to 40.3 . The profile is honest because a is uncertain too. Expect some students to call the slice honest “because it uses the best a .”
- **1b.** a is the log-odds of death at $x = 0$ (1 mg/L), far below the data (49-77 mg/L). Good fits pivot around the LD50 ($x \approx 1.77$), so a steeper slope needs a more negative intercept. Hardest question in the Act; let groups argue it out.
- **2a.** $SE_1 = 2.63$, $r = 0.433$, $SE_2 = 2.91$; intervals 29.1 to 39.4 and 28.6 to 40.0. SE_2 matches the profile. Common errors: sign of r (doesn’t affect SE_2), picking SE_1 because it’s smaller.
- **2b.** Uncentered: $SE_1 = 0.0735$ (reproduces the slice), $SE_2 = 2.91$ (same as centered). Centering changed the surface and the meaning of the intercept, not the fit. Rounding r to -0.9997 gives SE_2 near 3.0.

Act 2: An Interval for the Lethal Dose (laptop, ~80 min)

- **1a.** $L = -a/b \approx 1.772$, $LD50 = 10^L \approx 59.1$ mg/L. Watch for e^L (gives about 5.9).
- **1b-1c.** Binomial NLL with $p_i = 1/(1 + e^{-b(x_i - L)})$; pseudocode should re-optimize b for each candidate L (not hold it at $\hat{b}$).
- **2a.** Check value: $NLL(1.75, 30) \approx 32.19$ with `dbinom()`. $\hat{L} = 1.7717$, $SE = 0.0039$, Hessian interval 1.7642 to 1.7793. No sign flip on the Hessian (`optim()` minimized).
- **2b.** Profile interval about 1.7640 to 1.7791, nearly identical to 2a.
- **2c.** 59.1 mg/L, 58.1 to 60.1 mg/L.
- **Most likely coding bug:** a profile loop that re-estimates L instead of holding it fixed.

PART 2: PODIUM AND CIRCULATING CHEAT SHEET

Terse on purpose. Full rationale and code: Part 1.

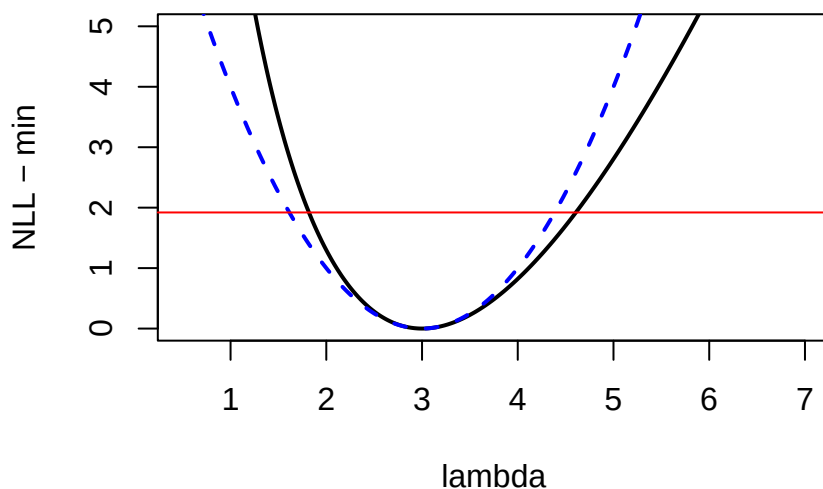
Opening ~25 min (content ~15). Act 1 ~45. Act 2 ~80.

- Don't derive the Poisson second derivative (Wed. take-home)
- Don't compare $1/\sqrt{H_{aa}}$ with `solve()` SEs (Act 1, 2a)

A. Quadratic approximation (~8 min)

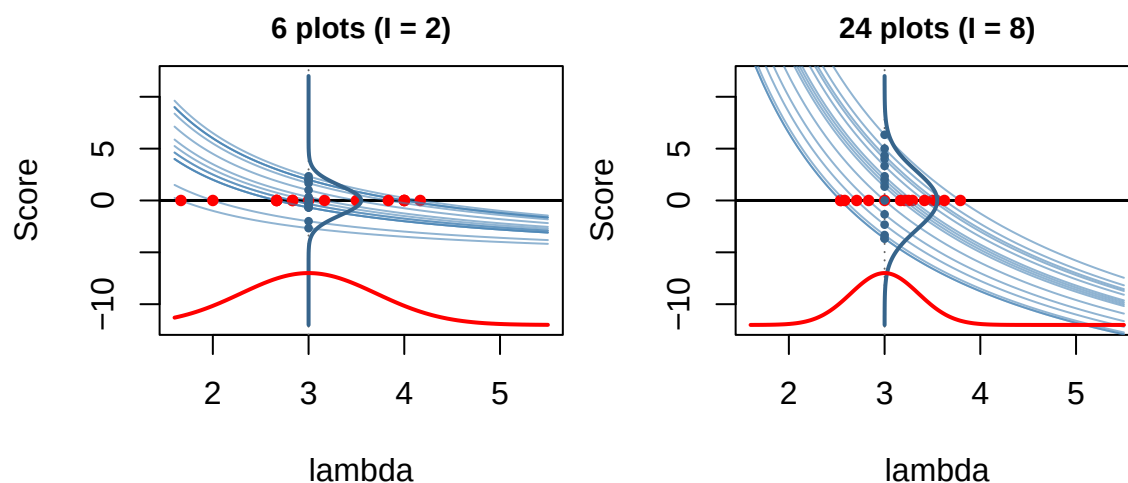
Pikas: 2, 5, 3, 0, 4, 4; MLE 3; LR [1.82, 4.61]

$$\begin{aligned} \text{NLL}(\lambda) &\approx \text{NLL}(\hat{\lambda}) + \frac{1}{2} \text{NLL}''(\hat{\lambda})(\lambda - \hat{\lambda})^2 \\ \Rightarrow \hat{\lambda} \pm \frac{1.96}{\sqrt{\text{NLL}''(\hat{\lambda})}}, \quad \text{SE} &= \frac{1}{\sqrt{\text{curvature}}} \end{aligned}$$



- `optim(hessian = TRUE)`: curvature 2, SE 0.71
- Hessian [1.61, 4.39] vs. LR [1.82, 4.61]
- Practical default in multiparameter software; LR = the check
- **Sticking point:** why the *second* derivative? Narrow vs. wide parabola

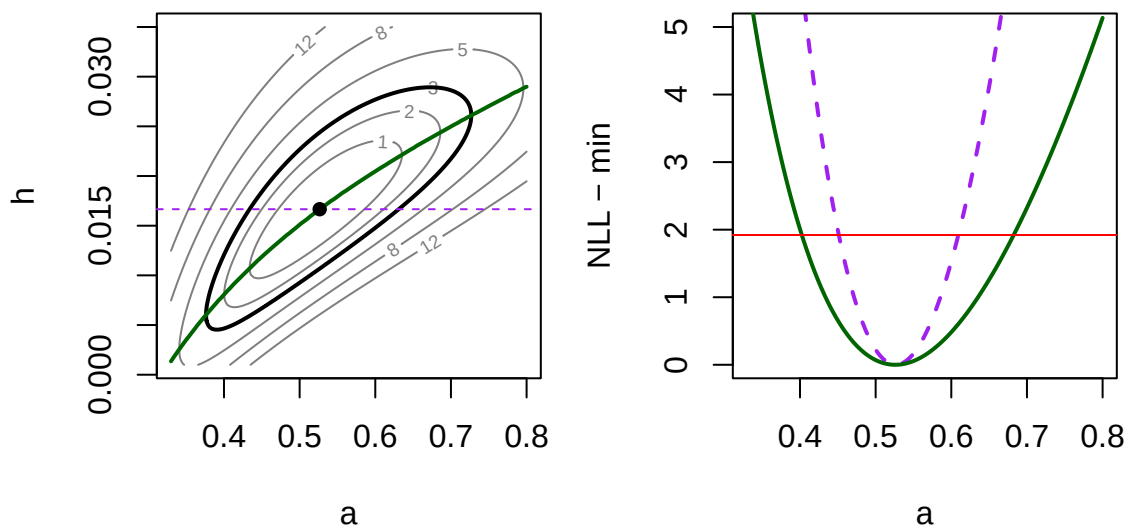
Aside (optional): why normal?



- MLE = where the score crosses 0
- Heights at the truth: sum over plots $\rightarrow$ normal (CLT), variance I
- Shift = height $\div$ slope; slope $\approx I \rightarrow$ variance $1/I$
- More data: heights spread like $\sqrt{n}$, slope grows like n
- Bent curves (small n) $\rightarrow$ lopsided crossings

B. More than one parameter (~7 min)

Tadpoles, $p = a/(1 + ahN)$



- Tilted valley = correlated estimates (cor 0.82)
- Slice: other parameter fixed. Profile: re-optimize it
- “Which would you trust?” Leave open (Act 1, 1a)
- `solve(hessian)` = variance-covariance matrix

Act 1 quick reference

- 1a: slice 34.1-34.4; profile 28.9-40.3; profile honest
- 1b: $a = \log\text{-odds}$ at 1 mg/L; fits pivot near $x \approx 1.77$
- 2a: SE_1 2.63, r 0.433, SE_2 2.91; 29.1-39.4 vs. 28.6-40.0
- 2b: uncentered SE_1 0.0735 (= slice), SE_2 2.91; $r = -0.9997$ gives about 3.0

Act 2 quick reference

- 1a: L 1.772; LD50 59.1 mg/L ($e^L = 5.9$ is wrong)
- 2a: check 32.19; $\hat{L}$ 1.7717, SE 0.0039, 1.7642-1.7793
- 2b: profile 1.7640-1.7791
- 2c: 59.1 (58.1-60.1) mg/L
- Bug: profile loop re-estimating L